A.J. INSTITUTE OF ENGINEERING & TECHNOLOGY, MANGALURU
Department of MCA

SOFTWARE DESIGN SPECIFICATION (SDS)

Applicable for ML projects

On

AI-ENABLED FACULTY ATTENDANCE, LEAVE AND SUBSTITUTE MANAGEMENT ERP WITH NLP-BASED LEAVE CLASSIFICATION AND ATTENDANCE ANALYTICS

Student Name:	**Shashank Poojary**
USN:	_____
Department:	Department of MCA
Program / Degree:	Master of Computer Applications (MCA)

1. System Design

1.1. Architectural Design

1.1.1 System Context Diagram

The System Context Diagram defines the operational boundaries and interfaces between external actors (Faculty, HOD, Admin), physical biometric hardware, academic text corpora, and downstream reporting engines.

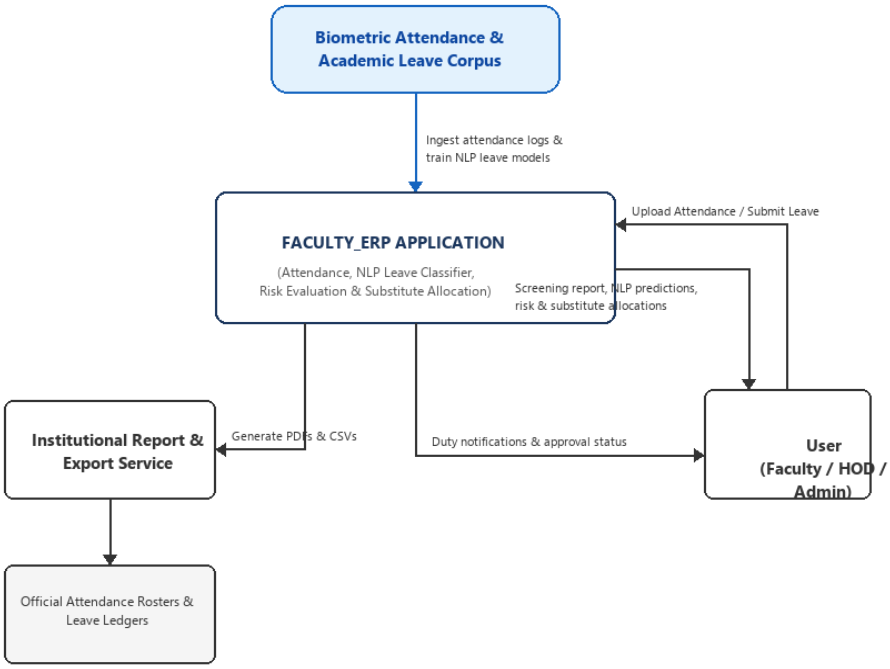


Figure 1.1: System Context Diagram showing Actors, Core ERP, Hardware & Services

1.1.2 End-to-End ML Pipeline

The end-to-end pipeline spans data ingestion, regex validation, text preprocessing, stratified 80:10:10 splitting, pure-NumPy model training, hyperparameter validation, model registry artifact storage, and real-time inference.

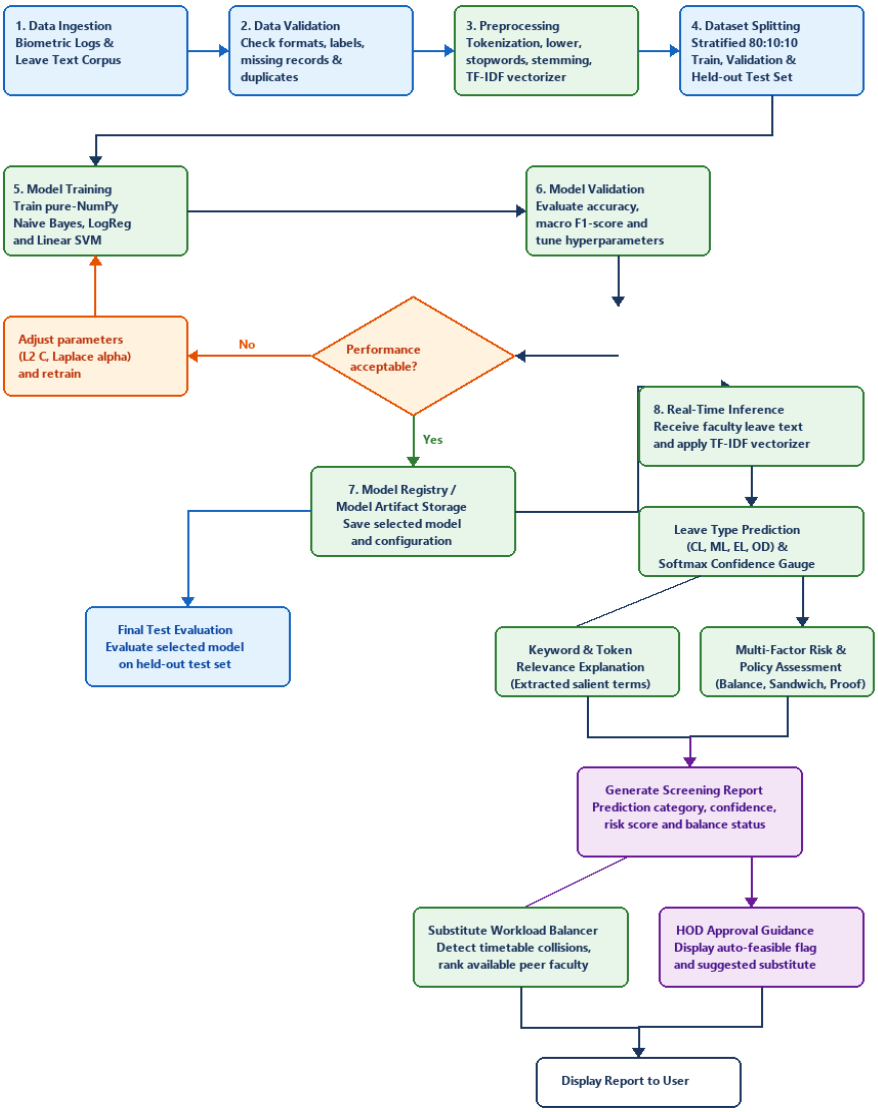


Figure 1.2: End-to-End ML Pipeline (Ingestion -> Validation -> Preprocessing -> Split -> Training -> Registry -> Inference)

2. Detailed Design

2.1 Dataset Design

2.1.1 Dataset Provenance and Permissions

Source: Optical/RFID biometric check-in terminals and a 500-sample balanced ground-truth leave reasoning corpus.
Owner & Clearance: Institutional Academic Administration; collected under operational research clearance with stripped PII.
Period & Intended Use: Academic Year 2024-2025; intended for administrative decision support, grace audit, and substitute balancing.

2.1.2 Dataset Schema and Data Dictionary

Field	Type	Meaning	Permissible Range	Missingness	Sensitivity
employee_code	String	Unique faculty employee ID	Alphanumeric (F001-F999)	Mandatory	Confidential
attendance_date	Date	Calendar punch date	Valid YYYY-MM-DD	Mandatory	Internal
login_time	Time	Morning biometric punch	07:00:00 to 12:00:00	Mandatory	Internal
leave_reason_text	Text	Free-text justification	10 to 500 characters	Mandatory	Confidential
leave_type_code	Categoric al	Target category	CL, ML, EL, OD, SCL, etc.	Mandatory	Confidential
has_medical_doc	Boolean	Medical proof flag	0 = No, 1 = Yes	Default 0	Confidential

2.1.3 Target / Label Definition

Multi-class target predicting 10 institutional leave categories: Casual Leave (CL), Medical Leave (ML), Earned Leave (EL), On Duty (OD), Special Casual Leave (SCL), Academic Vacation (AVL), Maternity (MAT), Paternity (PAT), Compensatory (CPL), and Restricted Leave (RL). Annotated by senior academic administrators with double-blind verification.

2.1.4 Partitioning and Leakage Prevention

Stratified 80:10:10 split (400 train [40/class], 50 val [5/class], 50 test [5/class]). Vocabulary building and IDF weights are frozen strictly on the training partition to prevent data leakage.

2.2. Preprocessing and Feature Engineering

2.2.1 Validation and Cleaning

Regex tokenization, lowercasing, non-ASCII stripping, and custom 68-word stopword filtering preserving domain tokens ('fever', 'doctor', 'conference', 'seminar', 'baby', 'accident', 'surgery').

2.2.2 Feature Selection / Extraction

Pure-NumPy TF-IDF engine: Sublinear TF: TF(t,d) = 1 + ln(count); Smoothed IDF: IDF(t) = ln((1+N)/(1+DF)) + 1; L2 normalization producing an 845-dimensional feature space.

2.3. Model Design

2.3.1 Baseline and Candidate Models

Three pure-NumPy classifiers evaluated: Multinomial Naive Bayes (Baseline, F1: 0.918), Linear SVM (Candidate, F1: 0.935), and Softmax Logistic Regression (Selected, Accuracy: 95.00%, Macro F1: 0.951). Selected model provides calibrated probabilities.

2.3.2 Model Architecture

Maps R^845 TF-IDF vector through linear affine transformation (z = W*x + b) and Softmax normalization to yield posterior class probabilities.

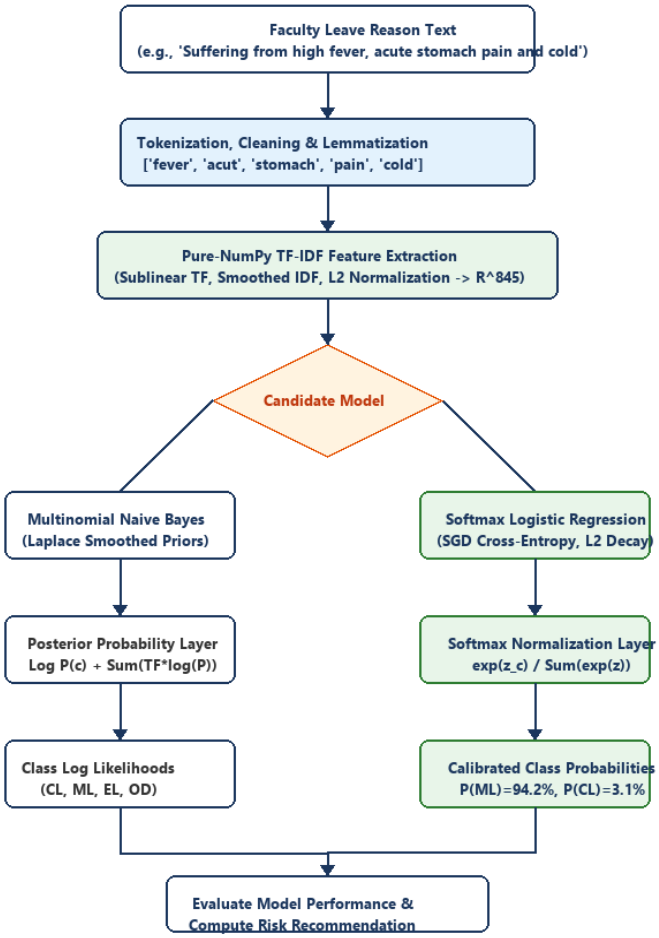


Figure 2.1: Model Architecture Diagram showing Token Vectorization and Softmax Inference

2.4. Database Design

2.4.1 ER Diagram / Storage Architecture

Relational MySQL 3NF database (12 tables) for transactions + local file store for serialized .joblib model artifacts + compressed .json.gz cold disaster recovery snapshots.

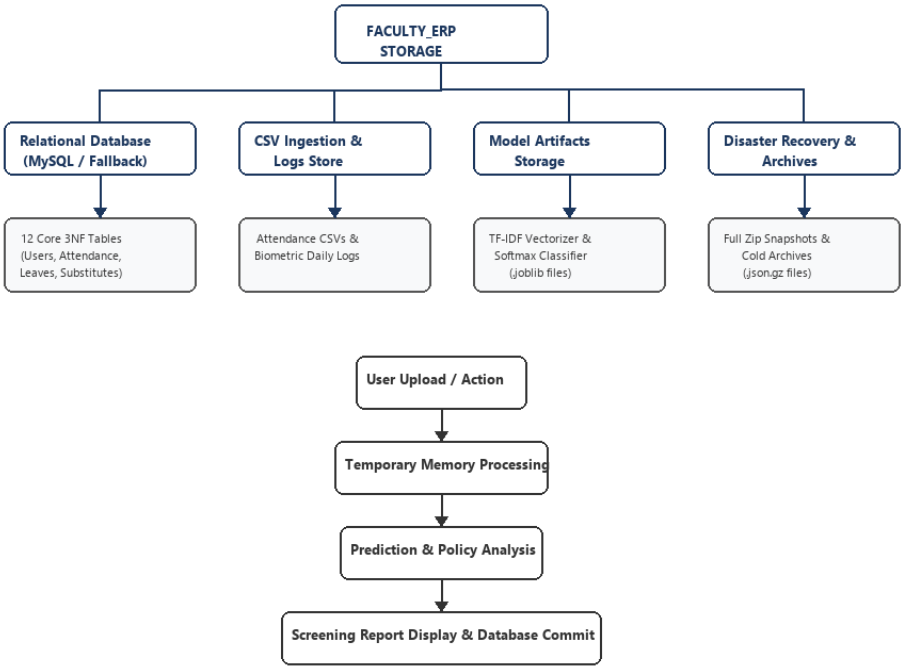


Figure 2.2: Storage Architecture showing Relational Database, File Store & Archives

2.4.2 Schema

Relational schema defines users (RBAC), departments, faculty_profiles, attendance, grace_minutes, leave_applications, substitute_assignments, and model_versions.

2.5. UML and Behavioral Design

2.5.1 Use Case Diagram

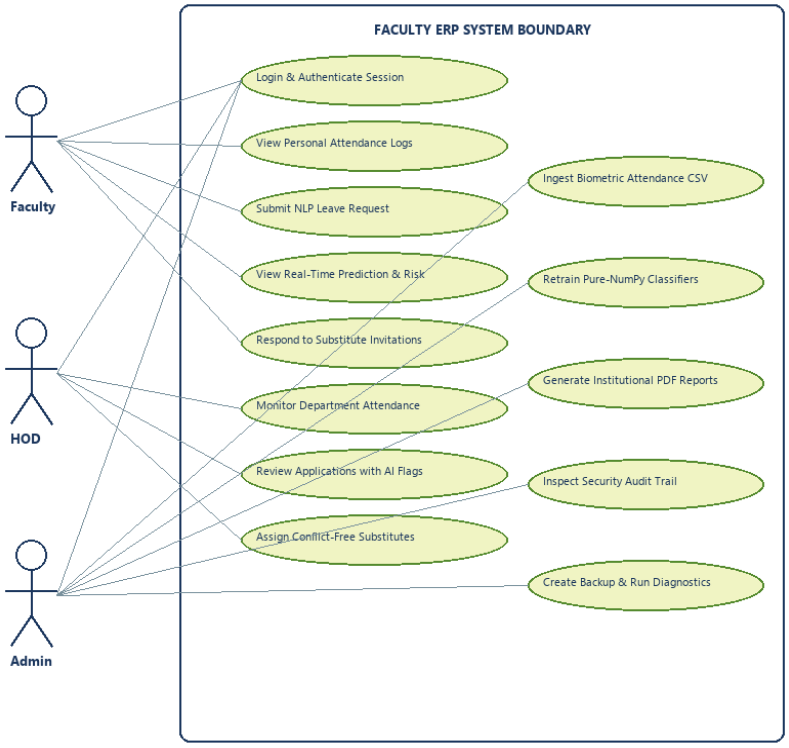


Figure 2.3: Use Case Diagram for Faculty, HOD, and Admin Roles

2.5.2 Class / Component Diagram

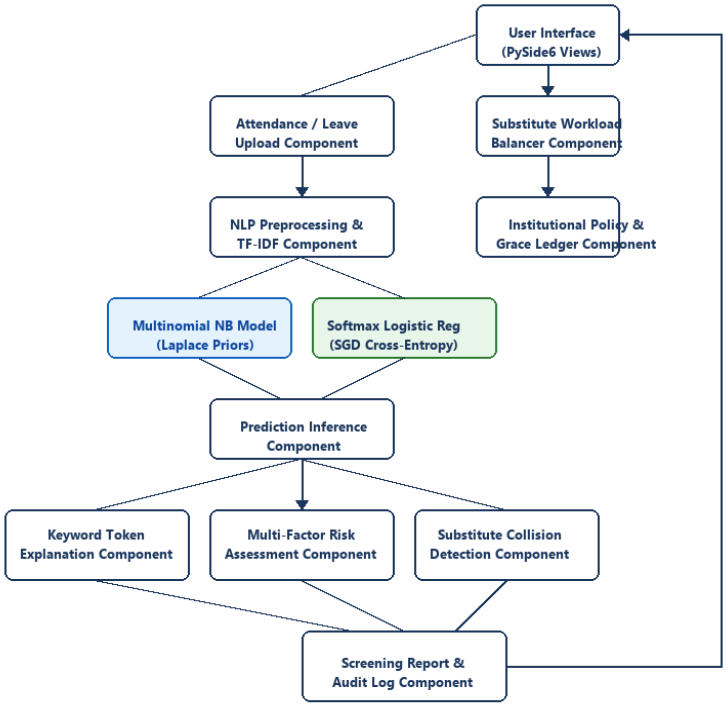


Figure 2.4: Class & Component Diagram showing Layered Architecture

2.5.3 Sequence Diagram

Captures the runtime flow from faculty request entry through preprocessing, inference, risk scoring, and database commitment.

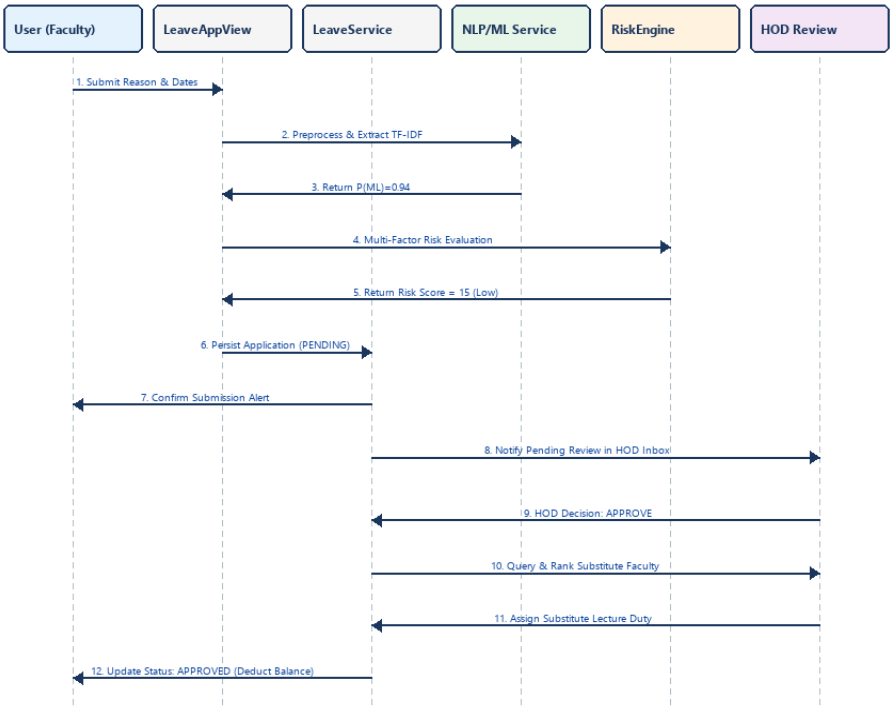


Figure 2.5: Sequence Diagram for Leave Inference & Decision Workflow

2.5.4 Activity Diagram

Illustrates the sequential activity path for model training, validation thresholds, parameter tuning, and real-time prediction routing.

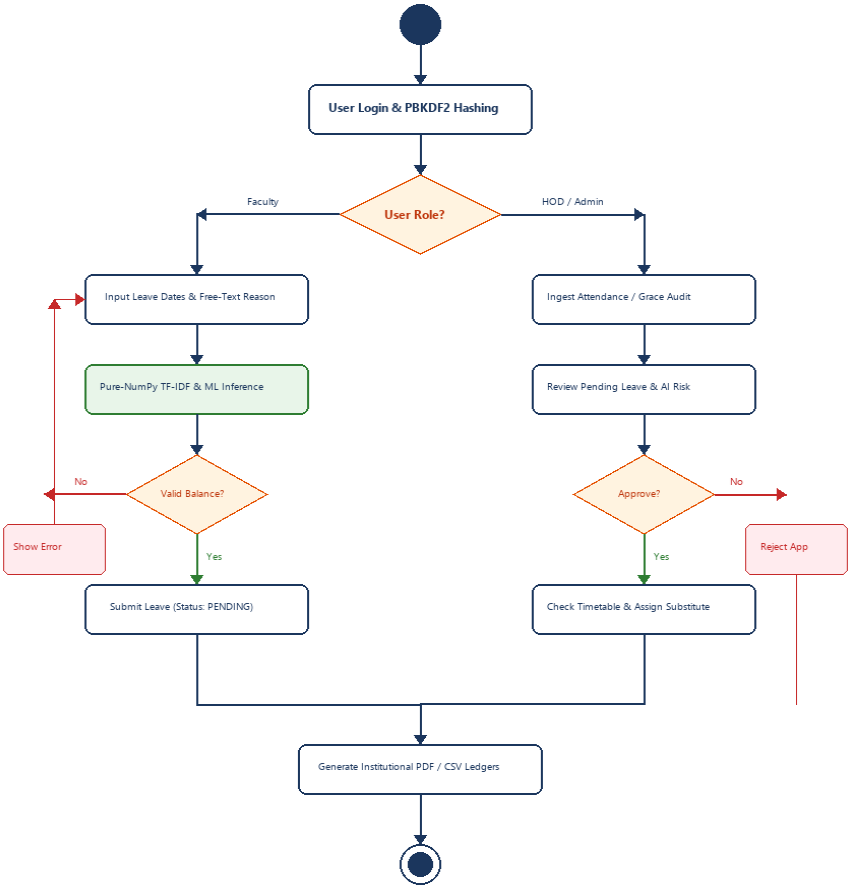


Figure 2.6: Activity Diagram displaying Training and Prediction Paths

2.6. User Interface Design

Desktop user interface built with PySide6 (Qt6), featuring interactive dashboards, login security, NLP leave screening, and substitute balancers:

2.6.1 Departmental Analytics Dashboard View

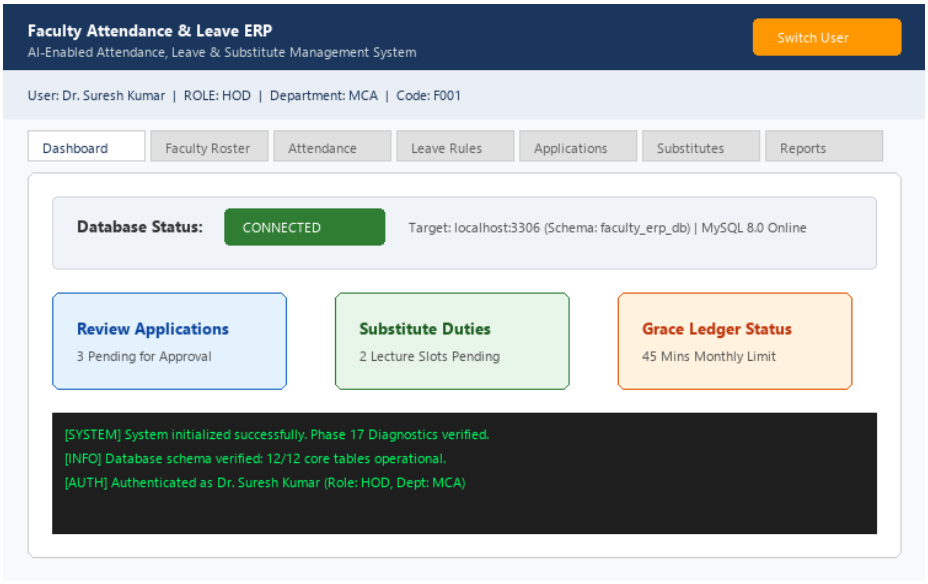


Figure 2.7: Departmental Analytics Dashboard View

2.6.2 Leave Application & AI Screening Engine View

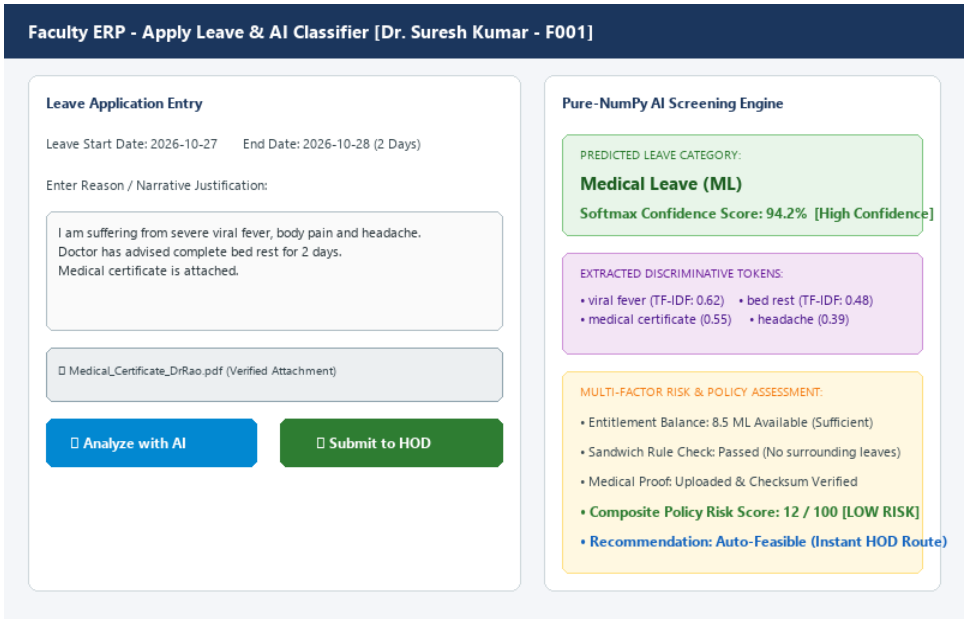


Figure 2.8: Leave Application View with Real-Time AI Screening

2.6.3 Substitute Duty Recommender & Workload Balancer View

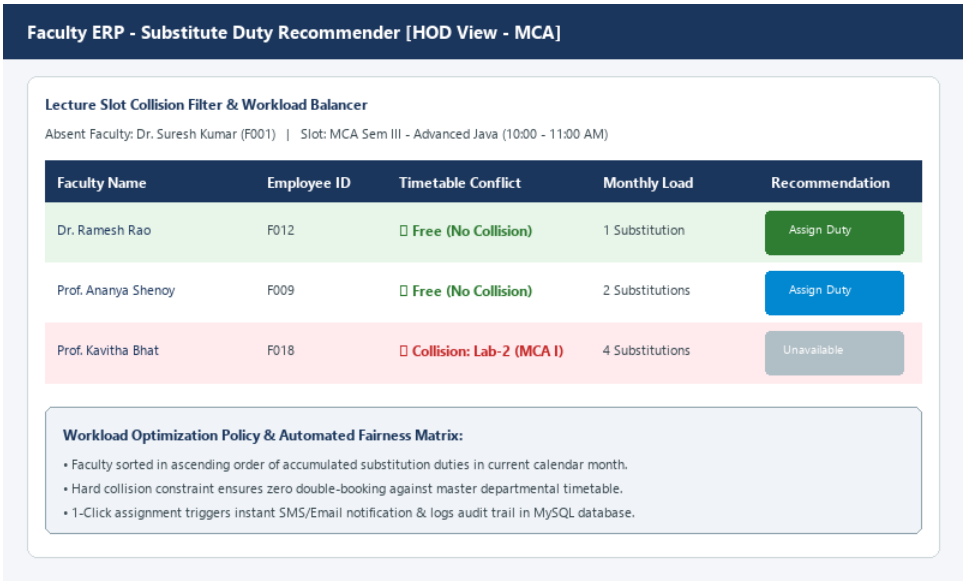


Figure 2.9: Substitute Duty Recommender View with Workload Balancing

2.6.4 Role-Based Authentication View

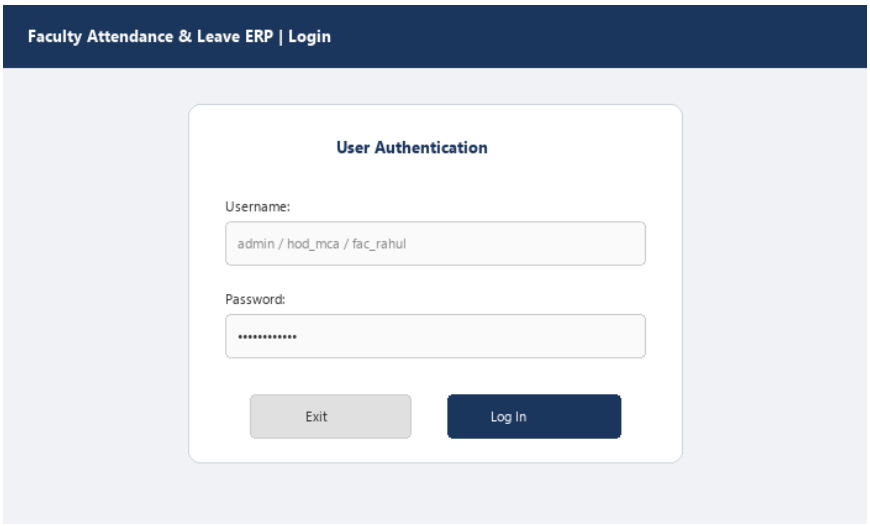


Figure 2.10: Role-Based Authentication Dialog

3. References

1. IEEE 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

2. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513-523.

3. McCallum, A., & Nigam, K. (1998). A comparison of event models for Naive Bayes text classification. AAAI-98 Workshop on Learning for Text Categorization, 752, 41-48.

4. Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273-297.

5. Python Software Foundation. PySide6 (Qt for Python 6) Framework Documentation. https://doc.qt.io/qtforpython-6/

6. ReportLab Inc. ReportLab PDF Generation User Guide (Version 4.0). https://www.reportlab.com/documentation/

7. Oracle Corp. MySQL 8.0 Reference Manual: Relational Database Storage Engines and InnoDB Architecture. https://dev.mysql.com/doc/refman/8.0/en/

8. Visual Paradigm & Lucidchart. UML 2.5 Specification: Use Case, Sequence, and Activity Diagram Standards. https://www.visual-paradigm.com/guide/uml-unified-modeling-language/

9. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

10. Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585(7825), 357-362.